



ARA0110 PADRÕES DE PROJETO

Aula 07: Padrões de projeto comportamentais - parte 2





RECAPITULANDO





PADRÕES COMPORTAMENTAIS

Os padrões de projeto comportamentais focam em algoritmos e atribuição de responsabilidades entre objetos, descrevendo padrões de comunicação entre eles.

Nesses padrões, encontramos uso intensivo de polimorfismo.

ITERATOR



DEFINIÇÃO

Fornecer um meio de acessar, sequencialmente, os elementos de um objeto agregado sem expor a sua representação subjacente.

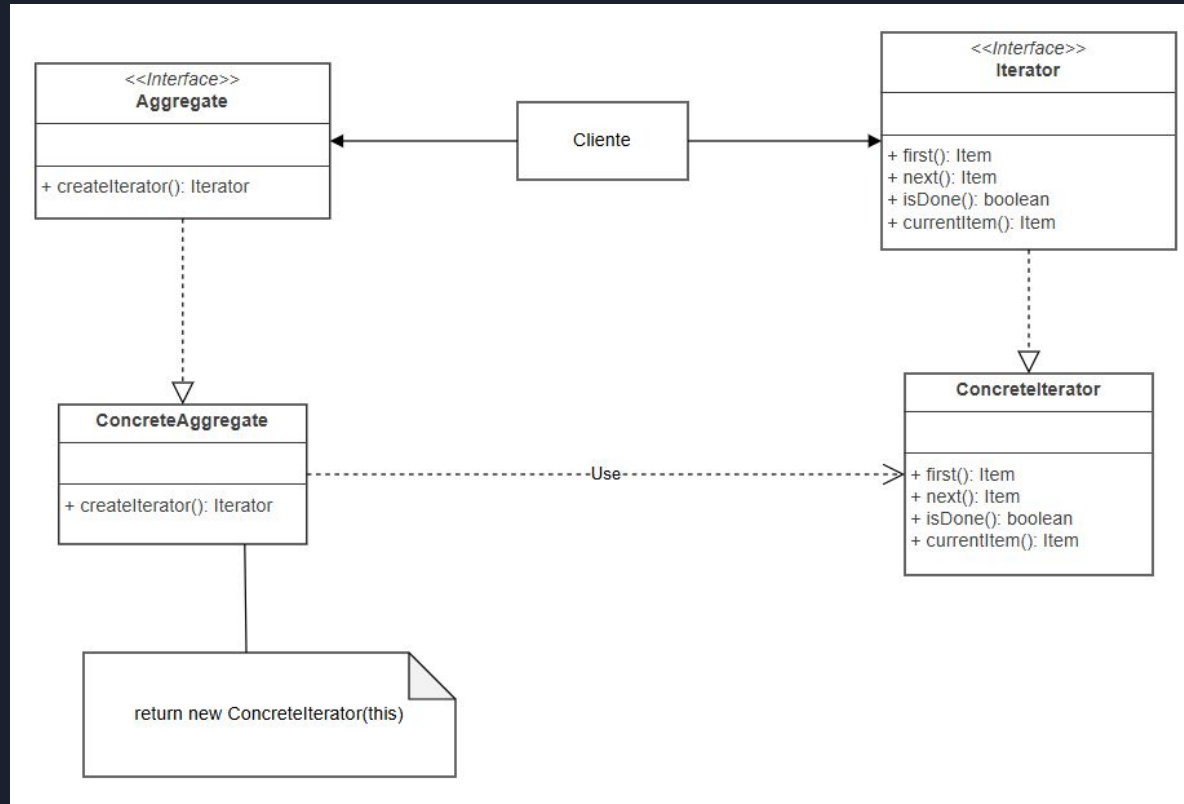
GoF, Padrões de Projetos. p.244

MOTIVAÇÃO

Imagine uma coleção complexa de dados na qual desejamos percorrer cada um dos itens. Para fazer isso, deveríamos conhecer a estrutura interna para saber onde a coleção começa e onde termina.

Com o padrão, contamos com um “guia especializado”: ele encapsula toda a lógica de travessia (como ir para o próximo, saber se acabou etc) em um objeto separado.

DIAGRAMA





PARTICIPANTES

- **Iterator:** interface que define operações para acessar e percorrer elementos.
- **ConcreteIterator:** implementação das operações de acesso e percurso, mantendo posição do agregado corrente.
- **Aggregate:** interface que define um padrão para criação do objeto Iterator.



PARTICIPANTES

- **ConcreteAggregate:** implementação da operação de criação de um Iterator apropriado.
- **Cliente:** elemento que interage com o agregado e o iterador de elementos.



UTILIZAÇÃO

O agregado define um padrão de criação de objetos Iterator de acordo com a dinâmica de interação com a coleção. Esse Iterator implementa e controla mecanismos para acesso e percurso de elementos, computando o objeto sucessor do percurso.



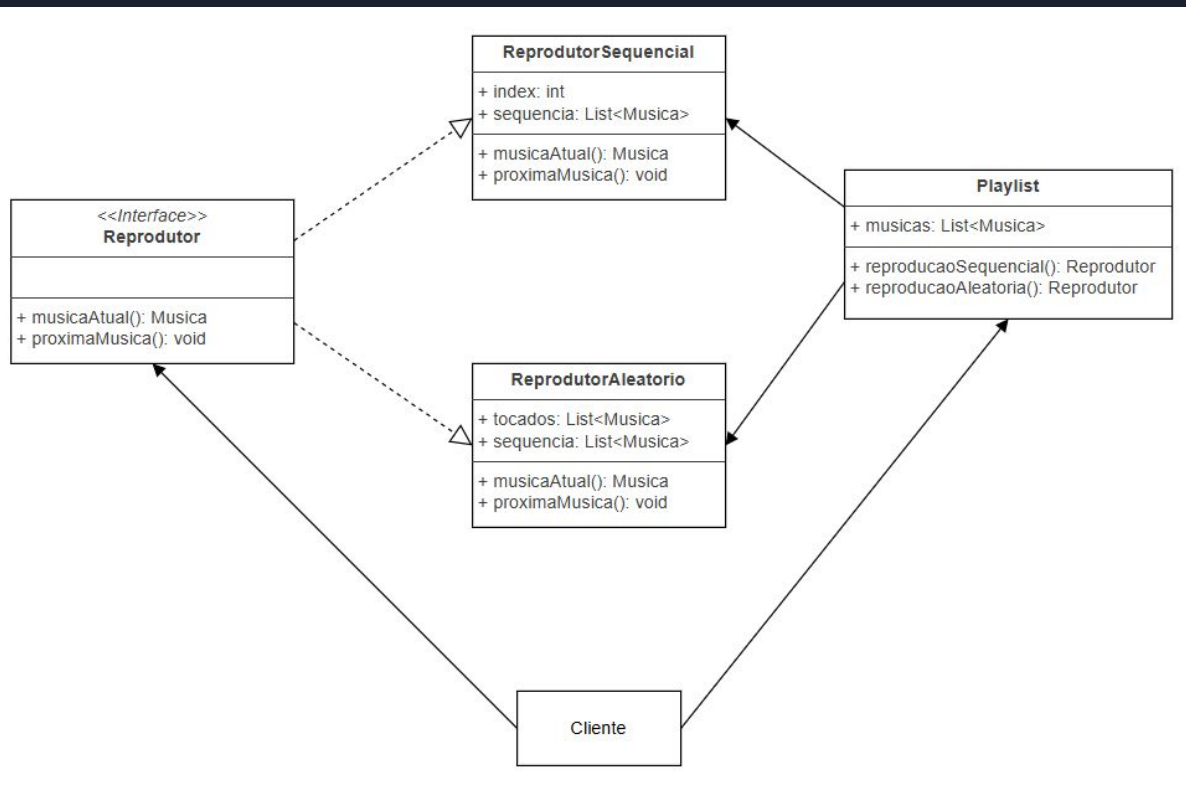
HORA DE EXERCÍCIO

Um aplicativo de streaming de música gerencia como uma playlist é executada:

- **Reprodução sequencial:** Segue a ordem alfabética dos nomes das músicas
- **Reprodução aleatória:** Não há uma ordem definida de reprodução, mas garante que não haja repetição de músicas na iteração

Vamos implementar esse aplicativo.

DIAGRAMA





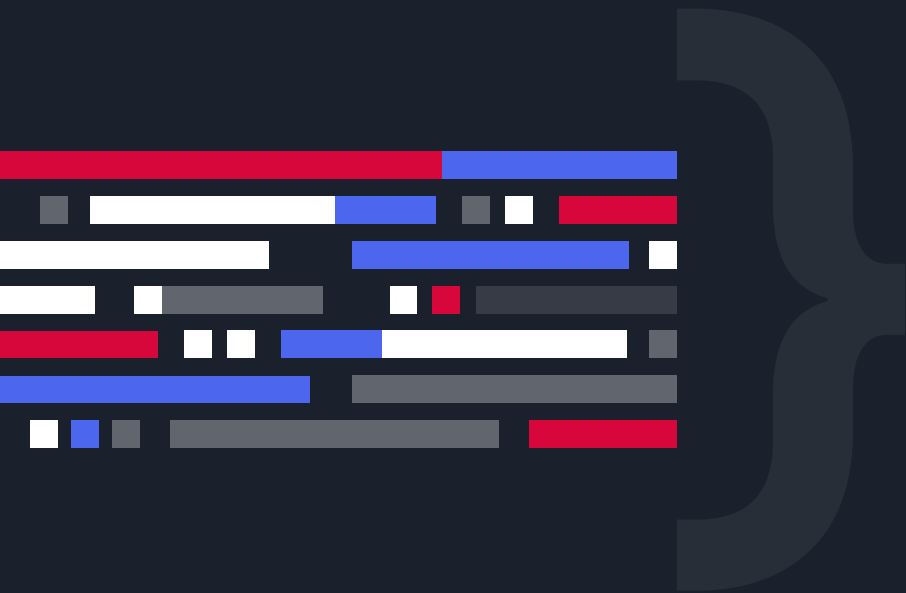
HORA DE EXERCÍCIO

No Instagram ou LinkedIn, podemos ter diferentes visualizações dos seguidores que possuímos, dependendo do contexto:

- Amigos próximos
- Amigos na mesma cidade
- Amigos em ordem alfabética
- Amigos em ordem de data de amizade

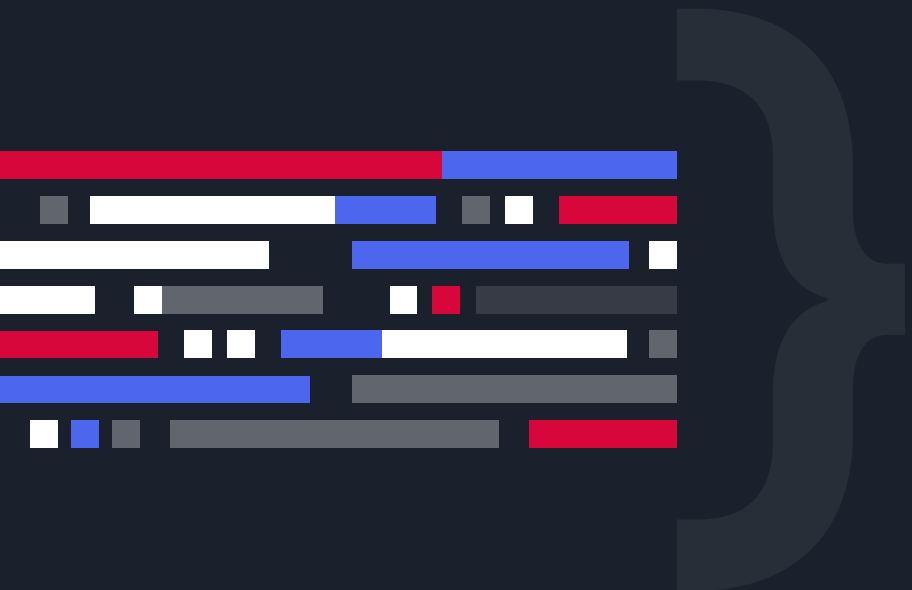
Vamos implementar esse sistema.

DIAGRAMA?



APLICABILIDADE

- Sistemas com diferentes visualizações de coleções de dados
- Sistemas de acesso a objetos agregados, sem expor sua representação interna
- Sistemas com diferentes estratégias de percurso para agregados de dados



CONSEQUÊNCIAS

- Oferecer suporte a variações no percurso de um agregado
- Iteradores simplificam a interface do agregado
- Mais do que um percurso pode estar em curso em um agregado



MEDIATOR





DEFINIÇÃO

Definir um objeto que encapsula a forma como um conjunto de objetos interage. O Mediator promove o acoplamento fraco ao evitar que os objetos se refiram uns aos outros explicitamente e permite variar suas interações independentemente.

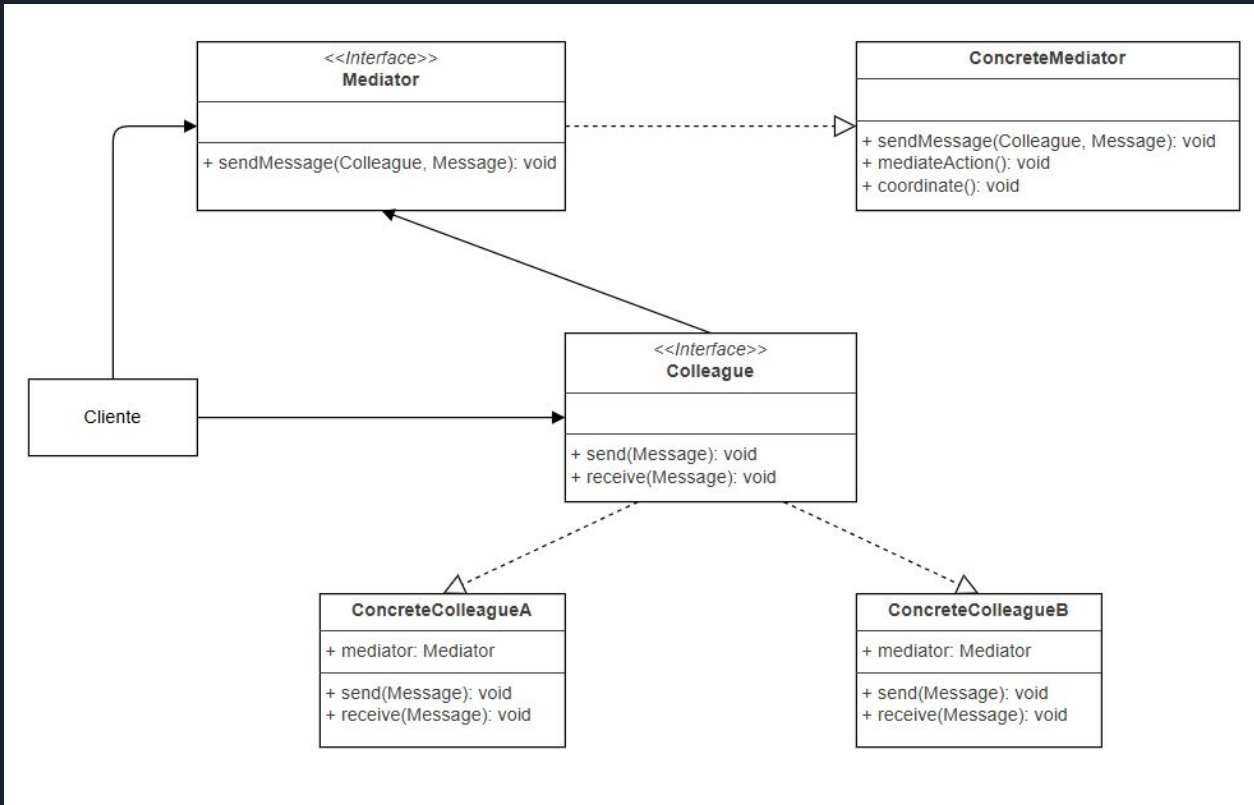
GoF, Padrões de Projetos. p.257



MOTIVAÇÃO

Imagine uma torre de controle de um aeroporto, onde os controladores mediam as interações entre pilotos para decidir pousos e decolagens. Em vez dos pilotos falarem uns com os outros, um controlador media essa interação, centralizando as decisões e coordenando o tráfego aéreo.

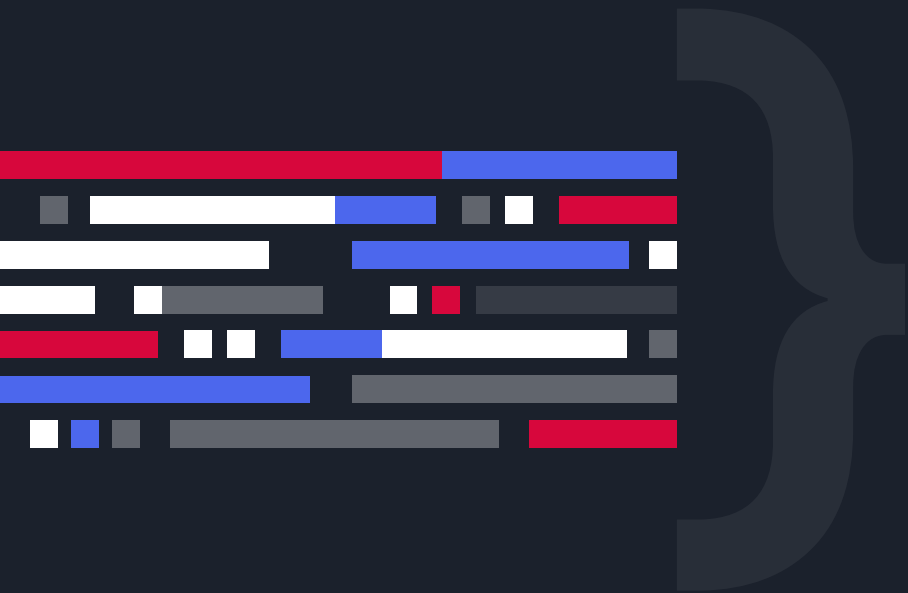
DIAGRAMA





PARTICIPANTES

- **Mediator:** interface que define o padrão de comunicação com objetos Colleague.
- **ConcreteMediator:** implementação da operação de comunicação dos objetos Colleague.
- **Colleague:** interface que define ponto de comunicação comum a partir do Mediator.



PARTICIPANTES

- **ConcreteColleagueX:** implementação de cada formato de comunicação com o Mediator, sem que haja interação direta com outro Colleague.
- **Cliente:** elemento que cria o padrão de comunicação e gerencia a comunicação entre os objetos Colleague.



UTILIZAÇÃO

Colegas enviam e recebem solicitações de um objeto Mediator criado. O comportamento cooperativo entre os objetos colegas para solicitações e respostas é implementado por um mediador, que coordena essa comunicação entre os objetos.

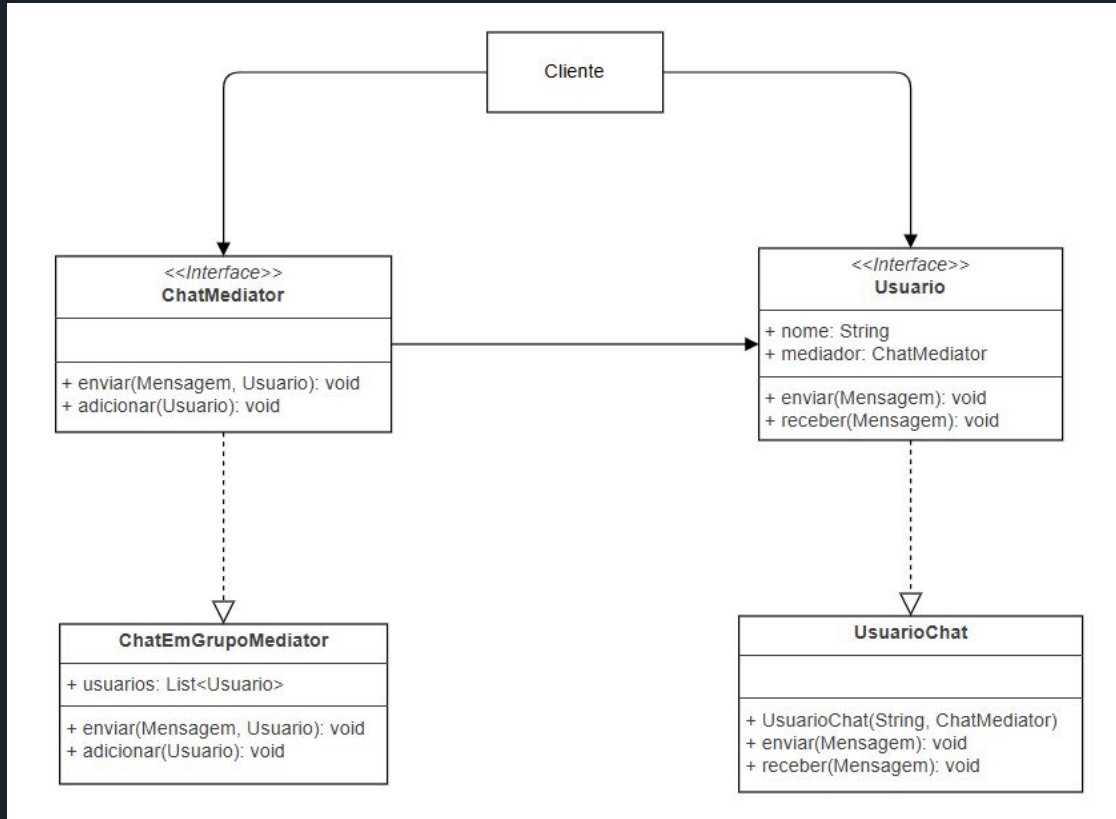


HORA DE EXERCÍCIO

Em um chat em grupo, um usuário não envia uma mensagem diretamente ao IP de cada colega presente na sala. O usuário envia a mensagem para o servidor, que ficará encarregado de entregá-la a todos os participantes da sala. Isso permite que novos usuários entrem e saiam sem ter necessidade de atualização interna nos usuários.

Vamos implementar esse sistema.

DIAGRAMA





HORA DE EXERCÍCIO

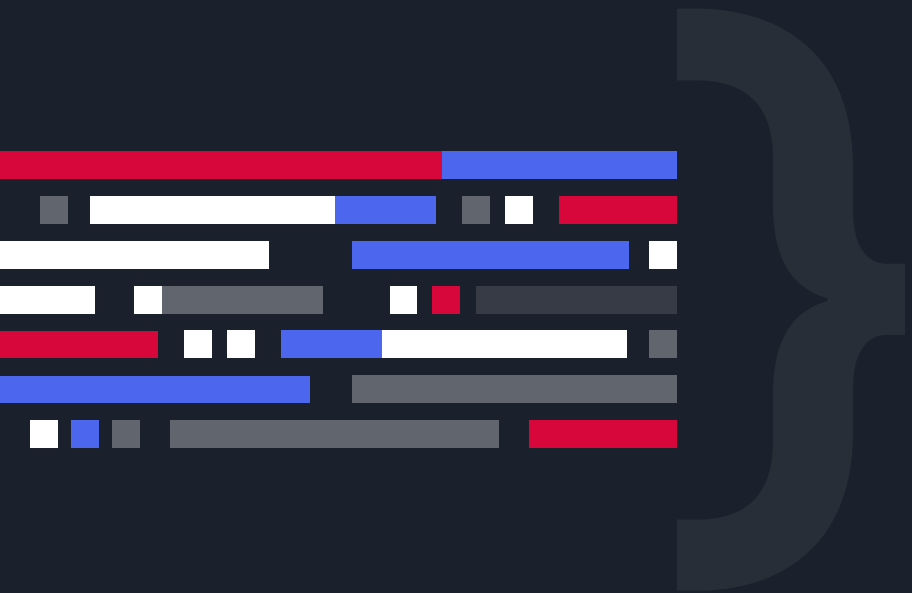
Considere um sistema de gerenciamento de robôs em um armazém automatizado. Para evitar colisões, os robôs não podem “adivinhar” para onde o outro está indo, então se faz necessário uma central que recebe as coordenadas de todos os robôs e gerencia, por comandos, quando um robô deve parar e outro deve seguir, garantindo fluidez sem que os robôs precisem se comunicar.

DIAGRAMA?



APLICABILIDADE

- Um conjunto de objetos precisa se comunicar de forma bem definida, mas complexa
- Um objeto se torna difícil de reutilizar quando se comunica com muitos objetos
- Um comportamento está distribuído entre várias classes que deveriam ser customizáveis ou adaptáveis



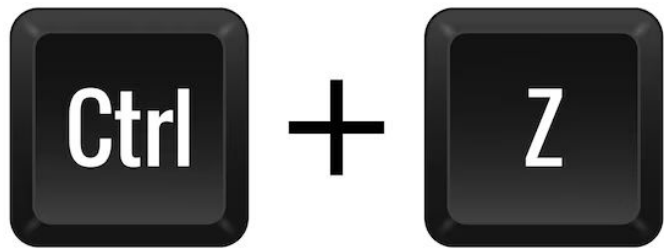
CONSEQUÊNCIAS

- Limitar o uso de subclasses
- Desacoplar objetos colegas
- Simplificar o protocolo dos objetos
- Abstrair a maneira como os objetos cooperam
- Centraliza o controle, logo, a complexidade da interação



MEMENTO





DEFINIÇÃO

Sem violar o encapsulamento, capturar e externalizar um estado interno de um objeto, de maneira que o objeto possa ser restaurado para esse estado mais tarde.

GoF, Padrões de Projetos. p.266

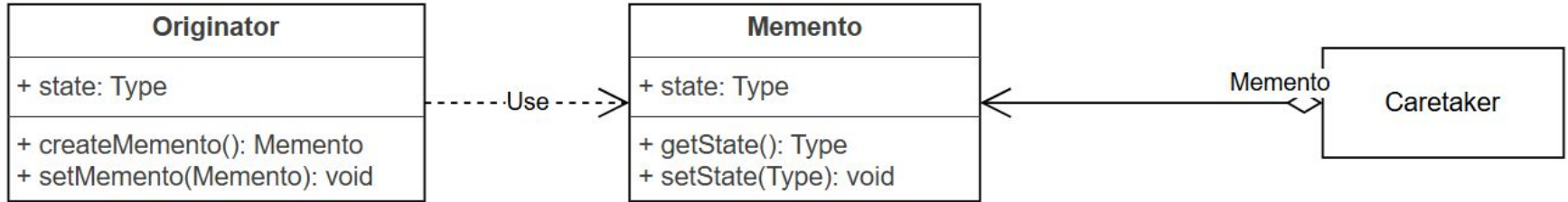


MOTIVAÇÃO

Em um editor de textos, como o Visual Studio Code, é comum recorrermos a funcionalidade de desfazer (`Ctrl + Z`) para voltar o código digitado a um estado anterior, em casos de erro de digitação ou mesmo utilização errada de componentes.

Dessa forma, o editor registra cada etapa do estado para que seja possível desfazer alterações e voltar a um estado anterior.

DIAGRAMA





PARTICIPANTES

- **Memento:** classe que armazena o estado interno do objeto Originator, protegendo acesso por objetos que não sejam o originator.
- **Originator:** classe que cria um objeto Memento contendo seu estado interno corrente, usando o mesmo para restaurar seu estado.
- **Caretaker:** elemento responsável pela custódia do Memento.



UTILIZAÇÃO

Um caretaker solicita um memento de um originador, mantém o mesmo por um certo tempo e o devolve para o originador.

Os objetos memento são passivos, ou seja, somente o originador que criou o memento será capaz de atribuir ou recuperar o seu estado.

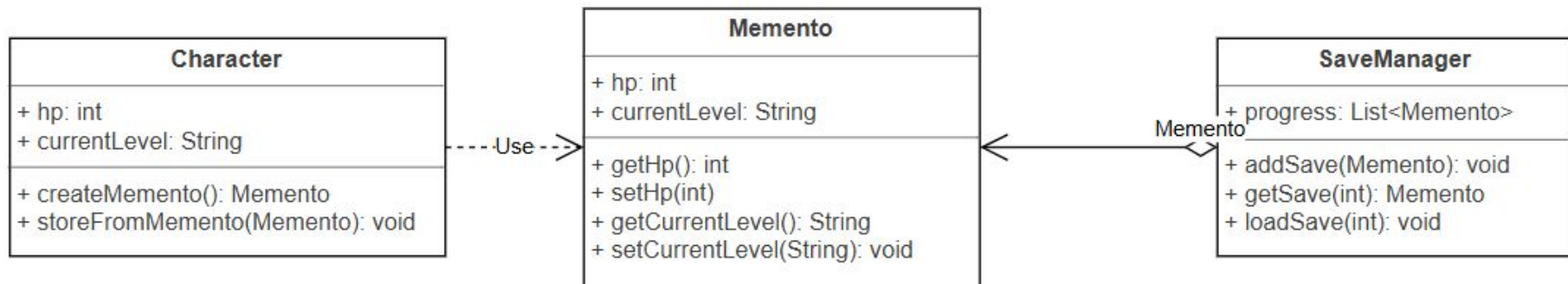


HORA DE EXERCÍCIO

Em jogos, é muito comum salvar o seu progresso ou o jogo criar um checkpoint em diferentes estágios, para salvar o progresso do jogador.

Vamos implementar um sistema de registro de progresso em um jogo.

DIAGRAMA



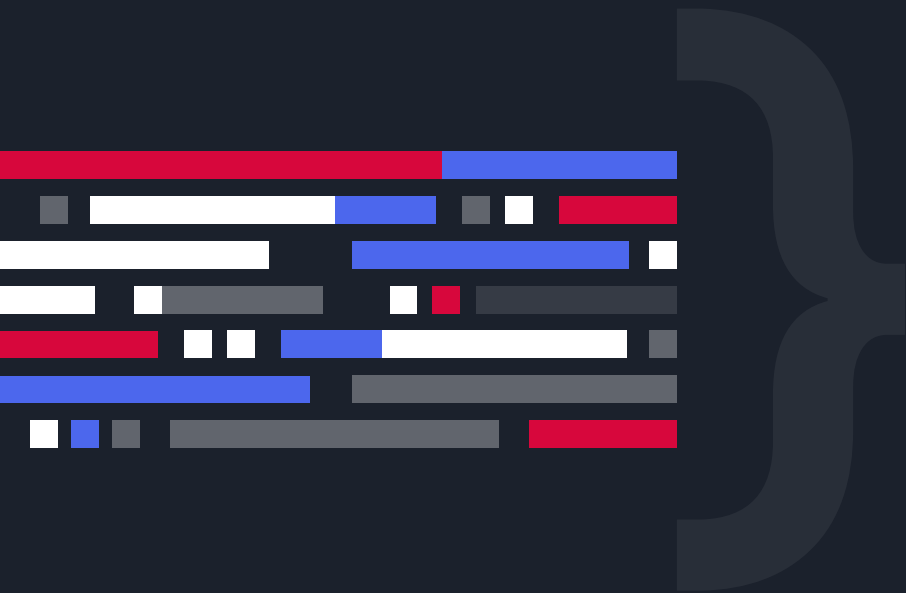


HORA DE EXERCÍCIO

Em sistemas de bancos de dados, cada operação que manipule um registro (criação, alteração ou remoção) é gerenciado através de uma transação. Dessa forma, se ocorrer algum problema durante o processo, o estado atual do registro pode ser restaurado através de um rollback.

Implemente um sistema de gestão de transações de banco de dados.

DIAGRAMA?



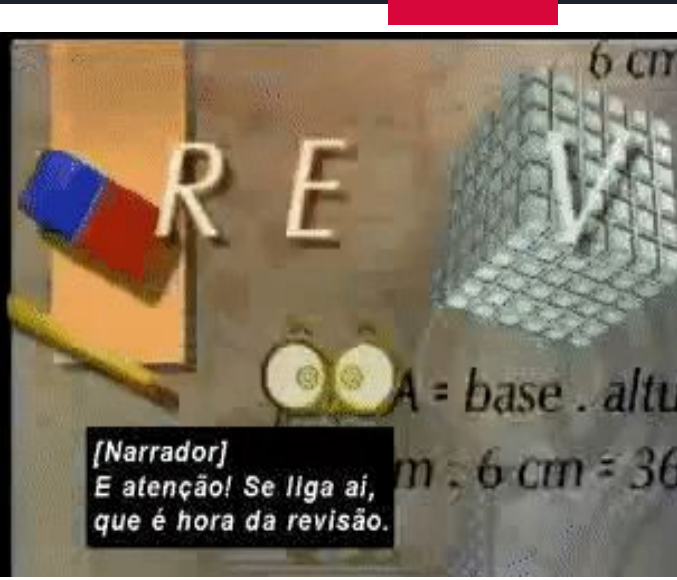
APLICABILIDADE

- Gerar um instantâneo de todo ou parte do estado de um objeto a ser salvo de modo que ele possa ser restaurado mais tarde
- Usar uma interface para obtenção do estado sem expor a estrutura interna do objeto, evitando romper encapsulamento



CONSEQUÊNCIAS

- Preservação das fronteiras do encapsulamento
- Simplificar o Originator
- Definir interfaces mínimas e amplas
- O uso de mementos pode ser computacionalmente caro
- Custos ocultos na custódia de mementos



RECAPITULANDO

- Padrões de projeto comportamentais
- Iterator
- Mediator
- Memento

OBRIGADO!



AGORA O VILGAX VAI SENTIR O GOSTO DO GRANDÃO!"

Ben Tennyson, a.k.a Ben 10